UNIVERSIDAD DEL VALLE

Escuela de Ingeniería de Sistemas y Computación

Moderando el extremismo de opiniones en una red social (ModEx)

Análisis y Diseño de Algoritmos II

Integrante: Grosman Klein Garcia Valencia

Profesor: Jesús Alexander Aranda

Monitor: Samuel Galindo

Noviembre de 2026

Resumen

En este proyecto se aborda el problema ModEx, que consiste en decidir qué agentes de una red social conviene moderar para reducir al máximo posible el extremismo de la red sin superar un presupuesto de esfuerzo. Se implementaron tres enfoques: fuerza bruta, un algoritmo voraz basado en la relación entre beneficio y costo, y programación dinámica. Al analizar la función objetivo, se observa que el problema se puede trabajar como una mochila 0/1: cada agente tiene un costo de moderación y un beneficio asociado a la reducción de la suma que determina el extremismo. Se compararon los tres algoritmos usando casos dirigidos y casos generados de forma reproducible. Los resultados muestran que fuerza bruta y programación dinámica encuentran la solución óptima, mientras que el enfoque voraz es mucho más rápido pero puede fallar en algunos casos por tomar una buena decisión local que bloquea una mejor combinación global.

1. Planteamiento del problema

Una red está formada por n agentes. Cada agente tiene una opinión entera entre -100 y 100, una receptividad entre 0 y 1 y una influencia entre 1 y 3. Moderar a un agente significa llevar su opinión a 0. El objetivo es escoger una estrategia de moderación que no supere el presupuesto R_max y que deje el extremismo de la red lo más bajo posible.

$$c_i = ceil(|o_i| * (1 - r_i) * inf_i)$$

El valor anterior representa el esfuerzo necesario para moderar al agente i. Si el agente no se modera, su costo dentro de la estrategia es 0. El extremismo de una red se calcula como:

$$Ext(RS) = \frac{\sqrt{\sum_{i=0}^{n-1} o_i^2 * inf_i}}{n}$$

1.1. Relación con mochila 0/1

Para resolver el problema fue útil notar que moderar un agente elimina de la suma interna del extremismo el término $o_i^2 * inf_i$. Por eso definimos como beneficio del agente:

$$beneficio_i = o_i^2 * inf_i$$

Como n es constante y la raíz cuadrada es una función creciente, minimizar el extremismo equivale a maximizar la suma de beneficios de los agentes moderados. De esta forma, cada agente se puede ver como un elemento de una mochila: su peso es el costo de moderarlo, su valor es el beneficio anterior y la capacidad de la mochila es R_max. Además, un agente solo puede ser seleccionado o no seleccionado, por lo que se trata de una mochila 0/1.

2. Solución por fuerza bruta

2.1. Descripcion

La solución por fuerza bruta parte de una idea directa: generar todas las estrategias posibles, revisar cuáles respetan el presupuesto y conservar la que produzca el menor extremismo. La estrategia se representa con un vector binario E de longitud n, donde 1 significa que el agente se modera y 0 que no.

```
E = 000...0
while existan estrategias:
    calcular esfuerzo(E)
    si esfuerzo(E) <= R_max:
        calcular extremismo después de moderar E
        si mejora la mejor solución:
            guardar E
    generar la siguiente estrategia binaria
```

En la implementación, la función next(...) hace avanzar el vector como si fuera un número binario. La función modexFB(...) calcula el esfuerzo, crea una copia de la red con las opiniones seleccionadas en 0 y compara el extremismo obtenido con el mejor valor encontrado hasta ese momento.

2.2. Complejidad

Con n agentes existen 2^n estrategias posibles. Para cada estrategia se recorre la red para calcular el esfuerzo y, cuando la estrategia es aplicable, también se recorre para calcular el nuevo extremismo. Por esta razón el tiempo es O(n * 2^n). El espacio auxiliar usado por una estrategia y por la copia de la red es O(n).

2.3. Correccion

La fuerza bruta siempre encuentra una solución óptima porque no descarta ninguna estrategia posible. Entre las 2^n combinaciones aparecen todas las estrategias aplicables y también la estrategia óptima. El algoritmo compara el extremismo de todas las estrategias que cumplen el presupuesto y conserva la mejor. Por lo tanto, al terminar no puede existir otra estrategia aplicable con menor extremismo que no haya sido revisada.

3. Solución voraz

3.1. Criterio utilizado

Para el algoritmo voraz se buscó un criterio que diera prioridad a los agentes que ofrecen una mayor reducción potencial por cada unidad de esfuerzo. Para cada agente se calculan su costo, su beneficio y la relación beneficio/costo.

$$ratio_i = \frac{beneficio_i}{costo_i}$$

Los candidatos se ordenan de mayor a menor ratio. Después se recorren en ese orden y cada agente se selecciona si su costo cabe en el presupuesto restante. Los agentes con costo 0 se procesan primero. En caso de empate en el ratio, la implementación prioriza mayor beneficio y luego menor índice.

```
calcular costo, beneficio y ratio de cada agente
ordenar candidatos por ratio descendente
remaining = R_max
para cada candidato:
    si costo <= remaining:
        seleccionarlo
        remaining -= costo
```

3.2. Ejemplos y casos adicionales

Se ejecutaron los dos ejemplos del enunciado y cuatro entradas adicionales. Para saber si la salida voraz era óptima en los casos pequeños, se comparó con fuerza bruta y con programación dinámica.

Caso	Entrada resumida	E voraz	Esf.	Ext.	Optima
Ejemplo 1	(100,.5,1), (100,.1,1), (-10,.1,1), R=55	[1,0,0]	50	33.4996	Si
Ejemplo 2	(-30,.9,1), (20,.1,2), (50,.5,1), R=35	[1,0,1]	28	9.4281	Si
n=1	(7,.5,2), R=7	[1]	7	0	Si
Todos caben	(-100,.5,1), (100,.5,3), (1,0,1), R=201	[1,1,1]	201	0	Si
Empate	(10,.5,1), (10,.5,1), R=5	[1,0]	5	5	Si
Voraz falla	(5,.1,3), (5,0,2), (-5,0,2), R=20	[1,0,0]	14	3.3333	No

3.3. Contraejemplo del algoritmo voraz

El caso más claro donde falla el criterio voraz tiene tres agentes y presupuesto 20. Sus costos y beneficios son los siguientes:

Agente	Costo	Beneficio	Beneficio/costo
A1	14	75	5.357
A2	10	50	5.000
A3	10	50	5.000

El voraz escoge primero A1 porque tiene el ratio más alto. Después quedan 6 unidades de presupuesto, por lo que A2 y A3 ya no caben. Su beneficio total es 75 y el extremismo final es 3.3333. Sin embargo, la combinación A2 + A3 cuesta exactamente 20 y tiene beneficio 100, dejando un extremismo de 2.8868. Esto muestra que una buena decisión local no siempre lleva a la mejor combinación global.

3.4. Complejidad y corrección

Crear la lista de candidatos toma O(n), ordenar toma O(n log n) y recorrer la lista toma O(n). Por lo tanto, la complejidad total es O(n log n), con espacio O(n).

El algoritmo no es correcto en el sentido de garantizar siempre el óptimo. Puede ser óptimo en muchos casos, por ejemplo, cuando el presupuesto alcanza para moderar todos los agentes relevantes, cuando no existen combinaciones alternativas que mejoren la selección por ratio o cuando la selección voraz coincide con el subconjunto óptimo. Sin embargo, al ser un problema de mochila 0/1, el ratio por sí solo no tiene la propiedad necesaria para garantizar optimalidad en todas las entradas.

4. Solución por programación dinámica

4.1. Subestructura óptima

La programación dinámica aprovecha que la decisión sobre un agente deja un problema más pequeño del mismo tipo. Para los primeros i agentes y un presupuesto b, el agente i puede quedar por fuera o puede ser seleccionado. Si no se selecciona, la solución depende solamente de los primeros i-1 agentes con el mismo presupuesto. Si se selecciona, se paga su costo y queda un subproblema con los primeros i-1 agentes y presupuesto $b - c_i$.

Si la parte restante de una solución óptima no fuera óptima para ese subproblema, podría reemplazarse por una mejor y la solución original también mejoraría. Eso contradice que la solución original fuera óptima. Por esta razón el problema tiene subestructura óptima.

4.2. Definición de los subproblemas

Definimos dp[i][b] como el máximo beneficio que se puede obtener usando solamente los primeros i agentes y teniendo un presupuesto de hasta b. Los valores de i van de 0 a n y los de b de 0 a R_max, por lo que existen (n+1)(R_max+1) subproblemas.

$$dp[0][b] = 0$$

$$si\ c_i > b:\ dp[i][b] = dp[i-1][b]$$

$$si\ c_i <= b: dp[i][b] = max(dp[i-1][b], dp[i-1][b - c_i] + beneficio_i)$$

4.3. Llenado de la tabla

La tabla se llena de arriba hacia abajo. Para cada agente se recorren todos los presupuestos desde 0 hasta R_max. Primero se copia la opción de no tomar el agente y, si el costo cabe, se compara con la opción de tomarlo. El valor final del problema queda en dp[n][R_max].

```
para i = 1..n:
    para b = 0..R_max:
        dp[i][b] = dp[i-1][b]
        si costo[i-1] <= b:
            conAgente = dp[i-1][b-costo[i-1]] + beneficio[i-1]
            dp[i][b] = max(dp[i][b], conAgente)
```

4.4. Ejemplo pequeño

Para visualizar la idea se puede usar un ejemplo con tres agentes cuyos costos son 2, 3 y 4, beneficios 30, 50 y 70, y presupuesto 6. La tabla queda:

Agentes disponibles	b=0	b=1	b=2	b=3	b=4	b=5	b=6
Ninguno	0	0	0	0	0	0	0
A1	0	0	30	30	30	30	30
A1,A2	0	0	30	50	50	80	80
A1,A2,A3	0	0	30	50	70	80	100

Por ejemplo, dp[2][5] vale 80 porque al tomar A2, que cuesta 3 y vale 50, quedan 2 unidades de presupuesto. Con esas 2 unidades la mejor solución previa vale 30, así que se obtiene 30 + 50 = 80. En cambio, dp[3][5] se mantiene en 80 porque tomar A3 daría solamente 70.

4.5. Reconstrucción de la estrategia

Después de calcular el valor óptimo, es necesario saber qué agentes lo producen. La reconstrucción empieza en dp[n][R_max] y avanza hacia la fila 0. Si dp[i][remaining] es diferente de dp[i-1][remaining], significa que permitir el agente i mejoró el valor y por eso ese agente pertenece a la estrategia que se está reconstruyendo. Se marca con 1 y se resta su costo al presupuesto restante. Si los valores son iguales, se puede continuar sin ese agente.

```
remaining = R_max
para i = n..1:
    si dp[i][remaining] != dp[i-1][remaining]:
        estrategia[i-1] = 1
        remaining -= costo[i-1]
```

4.6. Correccion

La recurrencia compara exactamente las dos posibilidades que existen para cada agente: no usarlo o usarlo, siempre que su costo quepa. Por inducción sobre i, si la fila i-1 contiene los valores óptimos para todos los presupuestos, entonces cada celda de la fila i escoge la mejor solución entre las dos únicas alternativas posibles. Por lo tanto, la fila i también queda óptima. Como la fila 0 es correcta por definición, dp[n][R_max] contiene el beneficio óptimo del problema completo.

4.7. Complejidad en tiempo y espacio

La implementación recorre una tabla de (n+1)(R_max+1) celdas y realiza una cantidad constante de trabajo por celda. Por eso, usando A = R_max como capacidad, el tiempo es O(nA) y el espacio también es O(nA), ya que se conserva la tabla completa para poder reconstruir la estrategia.

El mayor costo posible de un agente es ceil(100 * 1 * 3) = 300. Por lo tanto, gastar más de 300n no aporta nada, porque con 300n alcanzaría para cubrir incluso el costo máximo de todos los agentes. Podemos considerar una capacidad útil A <= 300n. Entonces:

$$O(nA) <= O(n * 300n) = O(n^2)$$

La misma cota aplica al espacio de la tabla. Esta es la razón por la que, aunque el código se vea como O(n * R_max), el peor caso se puede expresar como O(n^2).

4.8. Análisis práctico para n = 2^k

Siguiendo el supuesto del enunciado de 3*10^8 operaciones por minuto y 4 bytes por celda, usamos la cota A = 300n. El número aproximado de operaciones es 300*n^2 y el espacio es 4*300*n^2 = 1200* n^2 bytes. Como n = 2^k, n^2 = 4^k.

$$tiempo\ (min) \sim= 300*4^k / (3*10^8) = 4^k / 10^6$$

$$espacio\ (bytes) \sim= 1200*4^k$$

k	n	Tiempo aprox.	Espacio aprox.	Interpretacion
8	256	0.07 min	75 MB	Comodo
9	512	0.26 min	300 MB	Comodo
10	1,024	1.05 min	1.17 GB	Aceptable
11	2,048	4.19 min	4.69 GB	Aceptable, pero ya pesado
12	4,096	16.78 min	18.75 GB	Límite / poco práctico
13	8,192	1.12 h	75 GB	Inaceptable en equipo común
14	16,384	4.47 h	300 GB	Inaceptable
15	32,768	17.90 h	1.17 TB	Inaceptable

Con estas aproximaciones, hasta k=10 el comportamiento es razonable. Para k=11 el tiempo todavía puede ser aceptable, pero la memoria ya ronda varios GB. En k=12 la tabla requiere cerca de 19 GB bajo el supuesto de 4 bytes por celda, por lo que el espacio empieza a ser el principal problema en un computador común. A partir de k=13 tanto el tiempo como la memoria hacen que la solución sea poco práctica. En la implementación actual, las celdas se guardan como double, que normalmente ocupa más de los 4 bytes supuestos por el ejercicio, por lo que el consumo real puede ser mayor.

5. Implementación e interfaz

El proyecto fue implementado en C++17. Las tres funciones principales se llaman modexFB, modexV y modexPD y devuelven una estructura con la estrategia, el esfuerzo total y el extremismo final. Se usó Qt Widgets para la interfaz gráfica.

5.1. Formato de entrada

La aplicación permite seleccionar un archivo de texto. La primera línea contiene n, las siguientes n líneas contienen opinión, receptividad e influencia separadas por comas y la última línea contiene R_max.

```
3
100,0.5,1
100,0.1,1
-10,0.1,1
55
```

5.2. Formato de salida

La salida contiene primero el extremismo final, luego el esfuerzo y después una línea por agente con 0 o 1 según la estrategia calculada. La interfaz muestra la entrada y la salida y permite escoger cuál de los tres algoritmos ejecutar. También se guarda el resultado en un archivo de texto.

```
33.499585
50
1
0
0
```

6. Casos de prueba

Los casos no se escogieron solamente al azar. Se diseñó cada uno para revisar una situación concreta del problema. En los casos pequeños se usó fuerza bruta como referencia del óptimo y se verificó que la programación dinámica llegara al mismo valor.

Caso	Tipo	Que se busca probar
C01	Ejemplo 1 oficial	Verificar salida del enunciado y optimalidad.
C02	Ejemplo 2 oficial	Incluye una estrategia no aplicable y verifica el óptimo.
C03	n=1	Caso mínimo y presupuesto exactamente igual al costo.
C04	R_max=0	Comprueba un agente de costo 0 por receptividad 1.
C05	Todos caben	Presupuesto suficiente, opiniones +/-100 e influencias 1 y 3.
C06	Ninguno cabe	Comprueba que la estrategia vacía puede ser la única aplicable.
C07	Opinion 0	Costo y beneficio 0; permite estrategias distintas con el mismo valor.
C08	Empate	Dos soluciones óptimas con el mismo costo y beneficio.
C09	Voraz falla	Construido para que una decisión local bloquee una mejor combinación.
C10	Frontera double	Caso diagnóstico de representación de punto flotante antes de ceil.
C11	n=20, semilla 2026	Caso mediano para comparar tiempos de los tres algoritmos.
C12	n=1000, semilla 2026	Caso grande para comparar voraz y PD; FB no es viable.

6.1. Resultados de calidad

Caso	Ext. FB	Ext. Voraz	Ext. PD	Voraz optimo
C01	33.499585	33.499585	33.499585	Si
C02	9.428090	9.428090	9.428090	Si
C03	0	0	0	Si
C04	2.500000	2.500000	2.500000	Si
C05	0	0	0	Si
C06	8.660254	8.660254	8.660254	Si
C07	0	0	0	Sí, estrategia distinta
C08	5.000000	5.000000	5.000000	Si, hay empate
C09	2.886751	3.333333	2.886751	No
C11	18.237016	18.237016	18.237016	Si

Caso	Ext. FB	Ext. Voraz	Ext. PD	Voraz optimo
C12	N/A	2.310944	2.310892	No

En C07 y C08 las estrategias binarias no son necesariamente iguales, pero el valor obtenido si lo es. Esto es importante porque una entrada puede tener mas de una solucion optima. En C09 se observa el fallo intencional del voraz. En C12 no se ejecuto fuerza bruta debido a que con 1000 agentes tendria que considerar 2^1000 estrategias; la PD se toma como referencia optima por la correccion demostrada antes.

7. Comparación experimental

Para comparar tiempos se compilaron las implementaciones con optimización -O2 y se midió solamente la ejecución de cada algoritmo, sin contar lectura del archivo ni interfaz. Como los casos pequeños terminan demasiado rápido, se usaron varias repeticiones y se reportó el promedio. En C11 la fuerza bruta se repitió 3 veces y los otros algoritmos 100 veces. En C12 Voraz y PD se repitieron 20 veces. Los tiempos dependen del equipo, por lo que estos valores sirven principalmente para comparar tendencias.

Caso	n	FB (ms)	Voraz (ms)	PD (ms)
C01	3	0.000395	0.000132	0.000300
C09	3	0.000290	0.000130	0.000218
C11	20	67.134418	0.000662	0.002146
C12	1000	N/A	0.041928	1.417285

La diferencia más fuerte aparece en C11. Con solamente 20 agentes, la fuerza bruta ya necesita alrededor de 67 ms, mientras que los otros dos enfoques están muy por debajo de 1 ms. Esto concuerda con el crecimiento exponencial de 2^n. En C12, con 1000 agentes, no se intentó fuerza bruta. El voraz fue aproximadamente 34 veces más rápido que PD en esta ejecución, pero PD obtuvo un extremismo ligeramente menor: 2.310892 frente a 2.310944.

Lo importante de la prueba es que el orden observado coincide con el análisis: fuerza bruta se deteriora muy rápido al aumentar n; el voraz es el más liviano; y la PD usa más trabajo que el voraz para conservar la garantía de optimalidad.

8. Ventajas y desventajas de los enfoques

Enfoque	Ventajas	Desventajas
Fuerza bruta	Siempre encuentra el óptimo; es simple de entender; sirve como referencia para casos pequeños.	O(n*2^n); deja de ser viable rápidamente cuando crece n.
Voraz	Muy rápido; O(n log n); implementación sencilla; usa poca memoria.	No garantiza el óptimo; depende de que las decisiones locales produzcan una buena combinación global.
Programación dinámica	Encuentra el óptimo; mucho más útil que FB para tamaños moderados; permite reconstruir la estrategia.	Usa O(n*R_max) tiempo y memoria; la tabla puede crecer demasiado cuando n y el presupuesto son grandes.

No existe un único enfoque que sea el mejor en todos los sentidos. La fuerza bruta es adecuada para verificar instancias pequeñas. El voraz es atractivo cuando se necesita una respuesta muy rápida y se

acepta que pueda no ser óptima. La programación dinámica es la mejor opción de las tres cuando se necesita garantizar el óptimo y el tamaño de la tabla todavía cabe en tiempo y memoria.

9. Conclusiones

El proyecto permitió comparar tres formas bastante diferentes de resolver el mismo problema. La fuerza bruta confirma el óptimo revisando todas las posibilidades, pero su crecimiento exponencial hace que solo sea útil para entradas pequeñas. El algoritmo voraz reduce mucho el tiempo al tomar decisiones usando beneficio/costo, pero el contraejemplo construido muestra que esa regla no garantiza la mejor combinación.

La programación dinámica aprovecha la subestructura óptima del problema y obtiene la misma respuesta óptima que fuerza bruta en las pruebas pequeñas, sin enumerar las 2^n estrategias. Su costo depende del número de agentes y del presupuesto, y en el peor caso puede verse como O(n^2) si se usa que el costo máximo por agente es 300. El principal límite práctico termina siendo el tamaño de la tabla, especialmente cuando n crece a potencias de dos.

Finalmente, los casos de prueba mostraron que no basta con comparar solamente la estrategia binaria: pueden existir estrategias diferentes con el mismo esfuerzo y el mismo extremismo. Por eso, para validar las soluciones se compararon principalmente la aplicabilidad, el esfuerzo y el valor final de extremismo.

Anexo A. Archivos principales del proyecto

Archivo	Funcion
modex.h	Estructuras Agent, Network y ModexResult, y declaraciones de las tres soluciones.
modexFB.cpp	Implementación de fuerza bruta.
modexV.cpp	Implementación del algoritmo voraz.
modexPD.cpp	Implementación de programación dinámica y reconstrucción.
modexUtils.cpp	Cálculo de esfuerzo y extremismo.
parser.cpp / parser.h	Lectura de entradas y formateo de salidas.
mainwindow.*	Interfaz gráfica en Qt Widgets.